



LISP

TEARS OF JOY

Part—10

Lisp has been hailed as the world's most powerful programming language. But only the top percentile of programmers use it, because of its cryptic syntax and academic reputation. This is rather unfortunate, since Lisp isn't that hard to grasp. If you want to be among the crème de la crème, this series is for you. This is the tenth and final article in the series that began in the June 2011 issue of *LFP*.

There was a joke back in the 80s, when Reagan's SDI (Strategic Defense Initiative) programme was in full swing, that someone stole the Lisp source code to the missile interceptor program—and to prove it, he showed the last page of code...

In our quest for supreme knowledge, through the past nine articles on the Lisp series, we've travelled far enough to discover that LISP does not stand for Lots of Irritating Superfluous Parentheses (I know, stranger!). In this epilogue, let's summarise in brief why hacking Lisp code could be a profound experience in your life.

Lisp's Functional Programming advantage

Functional Programming is the art of writing programs that work by returning values, instead of modifying things. Functional Programming also enables you to create fabulously powerful and very efficient abstract programs – it is a mathematical approach to programming. In math, for at least the last 400 years, functions have played a very central role. Functions express the connection between parameters (the ‘input’) and the result (the ‘output’) of certain processes. In each computation, the result depends, in a certain way, on the parameters. Therefore, a function is a good way of specifying a computation. This is the basis of the functional programming style. A ‘program’ consists of the definition of one or more functions. With the ‘execution’ of a program, the function is provided with parameters, and the result must be calculated. Writing code in a functional style guarantees that a

function does only one thing (returns a value), and is dependent on one thing (the parameters passed to it). This equips you to control side effects.

However, some side effects are almost always necessary for a program to actually do something. This means that you can't write a useful program that has the entirety of its code written in the functional style. In Article 8, where we talked about CLOS, I pointed out James Hague's assessment of functional programming, where he argues that, "100 per cent pure functional programming doesn't work. Even 98 per cent pure functional programming doesn't work. But if the slider between functional purity and 1980s BASIC-style imperative messiness is kicked down a few notches—say, to 85 per cent—then it really does work. You get all the advantages of functional programming, but without the extreme mental effort and unmaintainability that increases as you get closer and closer to perfectly pure."